

# The Beginner's Guide to Docker Multi-Stage Builds

Large Docker images slow down server deployments and consume costly cloud storage space. Multi-stage builds help you create tiny production images by keeping build tools separate from your final container.

### What you'll learn:

1. Define Your Builder Stage
2. Start a Fresh Runtime Stage
3. Copy Compiled Artifacts Only
4. Build and Verify Image Size

### Aman Raj Singh

Cloud Operations Engineer @ iManage

[linkedin.com/in/mramanrajsingh](https://linkedin.com/in/mramanrajsingh)

# 1

TIP 1 OF 4

## 1. Define Your Builder Stage

Start your Dockerfile with a full SDK image to compile your source code.

Assign a name to this initial stage using the 'AS' keyword.

This stage installs compilers, dependencies, and build utilities.

All heavy tools remain isolated inside this temporary environment.

```
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o main .
```

# Cloud & DevOps Daily

Docker Multi-Stage Builds: Shrink Your Image Size by 90%

Friday, September 18, 2026

Swipe to tip 2 >> [linkedin.com/in/mrmanrajsingh](https://www.linkedin.com/in/mrmanrajsingh)

# 2

TIP 2 OF 4

## 2. Start a Fresh Runtime Stage

Add a second 'FROM' statement below your build instructions.

Choose a tiny runtime base image like Alpine Linux or Distroless.

This step resets the image layer history completely clean.

Your final image will build only on top of this small base.

```
FROM alpine:3.19
WORKDIR /app
```

# Cloud & DevOps Daily

Docker Multi-Stage Builds: Shrink Your Image Size by 90%

Friday, September 18, 2026

Swipe to tip 3 >> [linkedin.com/in/mramanrajsingh](https://linkedin.com/in/mramanrajsingh)

# 3

TIP 3 OF 4

## 3. Copy Compiled Artifacts Only

Use the COPY flag '--from=' to point back to your builder stage.

Transfer only the compiled binary or built production static assets.

Leave behind raw source files, temporary caches, and compiler packages.

This step alone reduces final image footprint by up to 90%.

```
COPY --from=builder /app/main .  
EXPOSE 8080  
CMD ["/main"]
```

# Cloud & DevOps Daily

Docker Multi-Stage Builds: Shrink Your Image Size by 90%

Friday, September 18, 2026

Swipe to tip 4 >> [linkedin.com/in/mrmanrajsingh](https://linkedin.com/in/mrmanrajsingh)

# 4

TIP 4 OF 4

## 4. Build and Verify Image Size

Run the standard docker build command on your updated Dockerfile.

Use the docker images command to view the file size of the output.

Compare your multi-stage container size against traditional single-stage builds.

You will see a massive drop in total megabytes used.

```
$ docker build -t myapp:slim .  
docker images myapp:slim
```

# Cloud & DevOps Daily

Docker Multi-Stage Builds: Shrink Your Image Size by 90%

Friday, September 18, 2026

See Key Takeaways on next page >> [linkedin.com/in/mrmanrajsingh](https://linkedin.com/in/mrmanrajsingh)

## Key Takeaways

- + Use specific version tags for base images instead of using latest
- + Keep build stage dependencies separate from production runtime needs
- + Add a .dockerignore file to prevent sending unwanted files to Docker
- + Chain shell commands together using '&&' to minimize overall image layers
- + Place least frequently changed Dockerfile steps near the top of the file

## Watch Out For

- ! Forgetting to copy required configuration or environment files to the runner stage
- ! Copying the entire source folder into the final stage instead of build artifacts
- ! Using full development OS images for the final deployment container

**Follow for daily Cloud & DevOps guides!**

[linkedin.com/in/mrmanrajsingh](https://linkedin.com/in/mrmanrajsingh)